

AP-HP - ChatDPI

Projet industriel MAIN4

Victor Pezennec–Deutsch

Mathieu Laurent

Raphaël Yzermans

Alexis Duch

Tuteur académique : M. Ben Kabongo



Sommaire

I	Concepts et Définitions	3
1	Tokenization et Embeddings	3
2	Transformer et Grands Modèles de Langage (LLM)	4
2.1	Architecture Transformer	4
2.2	Fonctionnement général des LLM	4
3	Retrieval Augmented Generation (RAG) et Information Retrieval	4
3.1	Composant Retrieval	4
3.2	Bi-encodeur	5
3.3	Re-ranking et Cross-encodeur	5
3.4	Composant Generation	5
4	Métriques d'Évaluation	5
4.1	Précision, Rappel et F1-score à k	6
4.2	NDCG@k (Normalized Discounted Cumulative Gain)	6
II	Présentation du jeu de données	7
III	Retriever : Modèles, Expérimentations, Résultats	8
1	Description globale du pipeline	8
2	Retrievers testés	9
2.1	Configuration Expérimentale et Hyperparamètres	9
2.2	SPLADE (Sparse Lexical and Expansion Model)	9
	Principe théorique	9
	Formulation mathématique	9
2.3	SentenceTransformers : Modèles Denses (Bi-Encoders)	9
	Principe théorique et Mathématique	9
	Modèles implémentés	10
2.4	Méthode Graph-based (Centralité avec NetworkX)	10
	Principe théorique	10
	Formulation mathématique	10
	Résultats	10
3	Premier axe d'amélioration : la fusion hybride	11
3.1	Définition et Mathématiques	11
3.2	Poids optimisés selon les requêtes	11
3.3	Cross-Encoder (Le Reranking)	12
	Différence Bi-Encoder vs Cross-Encoder	12
	Impact attendu et Utilisation	12
3.4	Résultats	12
4	Second axe d'amélioration : optimisation des requêtes	13
4.1	Query expansion par datasets	13
4.2	Query expansion & rewriting via DSPy	14
	Résultats	15
4.3	HyDE	16

5	Analyse des Résultats et Justification des Choix Techniques	17
5.1	Résultats globaux	17
5.2	Analyse et interprétation	18
5.3	Évaluation du compromis Temps vs Performance (Latence)	19
	La dichotomie Dense/Sparse et le coût de l'architecture LLM.	20
	Le paradoxe computationnel de la Fusion Hybride.	20
	Le surcoût du Cross-Encoder.	20
IV Conclusion et perspectives		22
1	Conclusion du projet	22
2	Perspectives d'amélioration	22
3	Retour d'expérience	23
4	Organisation	23
Annexes		27

Remerciements

Tout d'abord, nous souhaitons remercier Christel GERARDIN de nous avoir permis de nous impliquer dans un projet aussi stimulant, aussi à l'état de l'art techniquement et à aussi fort enjeu malgré notre peu d'expérience.

Plus largement, nous tenons à remercier chaleureusement l'équipe RESUSI de l'AP-HP, en particulier Marco NAGUIB, Émile VALIER, Eliott LAKS et Christel GERARDIN, pour nous avoir intégrés à l'équipe, et pour l'aide apportée sur tous les problèmes que nous avons rencontrés durant nos expérimentations. Grâce à vous, nous avons pu appréhender tous les enjeux d'ordre médical ou technique qui entourent ce projet, et nous comprenons mieux le fonctionnement d'une équipe sur un tel projet.

Enfin, un grand merci à notre tuteur académique Ben KABONGO qui nous a guidés sur les outils à utiliser, et nous a éclairé sur beaucoup de problématiques liées aux technologies utilisées.

Introduction

Dans le domaine médical, les dossiers numérisés des patients peuvent être très volumineux avec beaucoup de documents. On compte une moyenne de 120 documents par patient dans le dossier informatisé, les cliniciens font face à une surcharge informationnelle critique qui expose à des omissions d'informations vitales (allergies, limitations thérapeutiques, antécédents majeurs). Cela a un impact direct sur la qualité et la sécurité des soins. Lors de la prise en charge de patients, notamment aux urgences, certaines situations nécessitent une prise de décision rapide. Ces prises de décisions peuvent varier d'un patient à un autre selon des contrindications, des anciennes pathologies, des allergies, ... C'est pourquoi pouvoir accéder au dossier médical de manière efficace permettrait d'améliorer le traitement des patients, la communication entre les différentes équipes, d'optimiser le temps de consultation et de réduire la charge cognitive des soignants en situation d'urgence.

Le projet RESUSI développe et valide cliniquement un agent conversationnel intelligent basé sur l'architecture RAG (Retrieval-Augmented-Generation) couplée à un grand modèle de langage. L'outil propose deux fonctionnalités innovantes : la génération automatique de résumés structurés et personnalisables du dossier patient avec traçabilité complète vers les documents sources, et une interface conversationnelle ("Chat DPI") permettant d'interroger le dossier en langage naturel. L'originalité du projet réside dans la co-conception avec les cliniciens, la garantie de traçabilité des sources pour éviter les hallucinations, et une méthodologie rigoureuse de validation clinique en conditions réelles d'utilisation.

L'objectif est de délivrer cet outil en premier lieu pour les équipes d'urgences de l'AP-HP puis dans un second temps d'étendre l'utilisation aux autres services.

À l'heure actuelle, il n'existe pas d'outil de ce type ayant bénéficié d'une validation clinique suffisante. Une étude de 2025 [1] comparant 30 études datant d'entre 2019 et 2024 a constaté que 87% des outils sont entraînés sur des documents en anglais, et 73% le sont sur des données venant d'institutions américaines. De plus, seulement 7% ont effectué une validation sur des données externes, et 20% ont effectué une analyse des défaillances. Enfin, l'évaluation des risques pour les patients n'a été réalisée que dans 3% des cas, et aucune analyse des biais n'a été faite.

Malgré l'existence de nombreux outils, aucun n'a été validé cliniquement et assure ne pas présenter de danger pour les patients. Dans ce contexte, le développement d'un tel outil et sa validation par des cliniciens représente un pas en avant pour la prise en charge des patients aux urgences de l'AP-HP, mais également une première validation d'un tel modèle à l'international.

I Concepts et Définitions

1 Tokenization et Embeddings

Un token est une unité élémentaire de texte manipulée par un modèle de langage. Selon la méthode choisie, un token peut correspondre à un mot entier, à un signe de ponctuation, à un sous-mot, voire à un caractère. La tokenisation désigne ainsi le processus qui convertit un texte brut en une suite de tokens [2].

La forme la plus simple de tokenisation consiste à segmenter le texte en mots en s'appuyant sur les espaces et la ponctuation. Toutefois, les systèmes modernes utilisent fréquemment une tokenisation en sous-mots, qui permet de limiter la taille du vocabulaire tout en conservant la capacité de représenter des mots rares ou inconnus. Par exemple, le mot *tokenization* peut être découpé en *token* et *ization*. Parmi les méthodes les plus répandues, on peut citer *WordPiece*, notamment utilisé dans *BERT* [3]. Les tokeniseurs de ce type cherchent un compromis entre granularité fine et couverture lexicale.

Un *n*-gramme est une séquence ordonnée de n tokens consécutifs. Les *n*-grammes sont souvent utilisés pour modéliser un contexte local, repérer des cooccurrences fréquentes ou représenter des expressions figées. Par exemple, un unigramme correspond à un token isolé, un bigramme à une paire de tokens, et un trigramme à une suite de trois tokens.

Une fois le texte tokenisé, il faut le représenter sous une forme numérique exploitable par les algorithmes. Les représentations les plus classiques sont les représentations sparses, comme le *Bag-of-Words* (BoW) [4] et le *TF-IDF* [5], qui décrivent un document à partir des termes présents et, dans le cas du *TF-IDF*, pondèrent ces termes selon leur fréquence dans le document et leur rareté dans le corpus. Ces représentations sont simples et efficaces pour certaines tâches, mais elles capturent mal la similarité sémantique fine entre les mots.

À l'inverse, les représentations denses, souvent appelées *embeddings*, associent à chaque mot, token ou séquence un vecteur de dimension réduite. Parmi elles, on distingue d'abord les *embeddings* statiques, comme *Word2Vec* et *GloVe*. Ces modèles apprennent des représentations distribuées à partir des contextes d'apparition dans le corpus d'entraînement, mais attribuent ensuite généralement un vecteur unique à chaque mot du vocabulaire, indépendamment du contexte d'utilisation. Ainsi, un mot polysémique comme « *souris* » reçoit en général la même représentation qu'il désigne l'animal ou le périphérique informatique.

Les modèles plus récents produisent au contraire des *embeddings* contextuels ou dynamiques. Dans ces modèles, la représentation d'un token dépend de son environnement dans la phrase. *BERT*, par exemple, construit des représentations bidirectionnelles tenant compte des mots situés à gauche et à droite du token considéré. Ce type d'encodage permet de mieux distinguer les différents sens d'un même mot selon le contexte. Par extension, certains modèles sont également conçus pour produire des *embeddings* de phrases ou de segments entiers, comme *Sentence-BERT* [6], afin de mieux capturer la similarité sémantique entre séquences.

2 Transformer et Grands Modèles de Langage (LLM)

Les Grands Modèles de Langage (LLM) sont des modèles de traitement du langage naturel basés sur l'architecture Transformer. Cette architecture a marqué une rupture majeure avec les approches précédentes basées sur les réseaux récurrents (RNN) ou convolutionnels (CNN), en proposant un modèle entièrement fondé sur des mécanismes d'attention, permettant un calcul parallèle efficace et évitant notamment les problèmes d'évanouissement du gradient rencontrés dans les RNN. Initialement, le Transformer a été proposé sous la forme d'une architecture encodeur-décodeur. Cependant, de nombreuses variantes ont émergé par la suite. Les modèles de type encodeur-only sont principalement utilisés pour la compréhension du langage naturel, comme BERT. Les modèles encodeur-décodeur sont adaptés aux tâches de transformation de texte comme la traduction ou le résumé, comme T5 ou BART. Enfin, les modèles décodeur-only sont utilisés pour la génération de texte, comme GPT, LLaMA ou Qwen. Les modèles encodeurs excellent dans les tâches de compréhension telles que la classification ou l'analyse de sentiment, tandis que les modèles décodeurs sont particulièrement performants pour les tâches de génération comme le résumé, la question-réponse ou la génération libre.

2.1 Architecture Transformer

Le Transformer repose sur le mécanisme de self-attention [7], qui permet au modèle de pondérer l'importance des différents mots d'une séquence afin de capturer le contexte global. Contrairement aux architectures précédentes, il ne traite pas les données de manière séquentielle comme les RNN, permet une parallélisation complète des calculs et capture efficacement les dépendances longues dans le texte. Chaque couche du Transformer est composée de mécanismes de self-attention multi-têtes, de réseaux feed-forward, ainsi que de mécanismes de normalisation et de connexions résiduelles.

2.2 Fonctionnement général des LLM

Selon leur architecture, les LLM peuvent encoder un texte en représentations vectorielles riches, générer du texte de manière auto-régressive ou combiner les deux approches. Dans les modèles décodeurs comme GPT, la génération se fait token par token en utilisant la self-attention pour considérer le contexte déjà généré ainsi que des mécanismes multi-têtes permettant de capturer différentes relations linguistiques.

3 Retrieval Augmented Generation (RAG) et Information Retrieval

Le Retrieval-Augmented Generation (RAG) [8] est une architecture combinant recherche d'information et génération de texte. L'objectif est d'améliorer les réponses d'un modèle de langage en lui fournissant des connaissances externes issues d'une base documentaire. Le pipeline général consiste à transformer une question utilisateur en vecteur, rechercher les documents les plus pertinents, appliquer un re-ranking puis générer une réponse.

3.1 Composant Retrieval

Le module de retrieval consiste à identifier les documents les plus pertinents dans une base de connaissances. Cette recherche peut s'appuyer sur des méthodes lexicales basées sur les mots

exacts, comme BM25 [9], ou sur des méthodes sémantiques basées sur des embeddings, comme Sentence-BERT [6]. Les méthodes sémantiques permettent de capturer les similarités de sens entre les textes, contrairement aux méthodes lexicales.

3.2 Bi-encodeur

Les bi-encodeurs encodent séparément la requête et les documents, puis comparent leurs embeddings à l'aide d'une mesure de similarité. Par exemple, Sentence-BERT [6] est un bi-encodeur basé sur le modèle BERT [3], et il existe une version conçue pour le français [10].

3.3 Re-ranking et Cross-encodeur

Après une première étape de retrieval, une phase de re-ranking est appliquée afin d'améliorer la pertinence des résultats. Le re-ranking consiste à utiliser un modèle plus précis pour réordonner les documents récupérés. Les cross-encodeurs sont particulièrement adaptés à cette tâche car ils encodent la requête et le document conjointement, ce qui permet de mieux modéliser leurs interactions. Ces modèles offrent une grande précision mais sont plus coûteux en calcul, ce qui les rend moins adaptés à une utilisation directe sur de très grands corpus sans étape préalable de filtrage.

3.4 Composant Generation

Une fois les documents pertinents sélectionnés et réordonnés, ils sont fournis au modèle génératif. Celui-ci produit une réponse en s'appuyant à la fois sur le contexte récupéré et sur ses connaissances internes. Des modèles comme Qwen ou LLaMA sont couramment utilisés dans cette étape. Cette approche permet de réduire les hallucinations et de fournir des réponses plus fiables et actualisées.

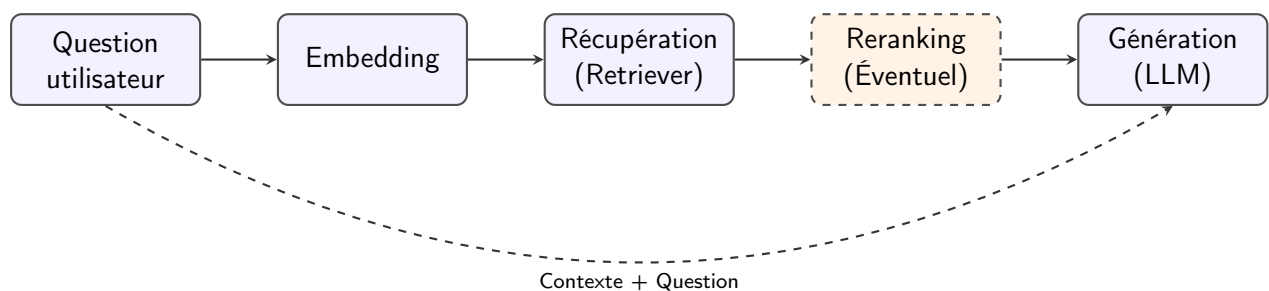


FIGURE I.1 – Pipeline RAG classique

4 Métriques d'Évaluation

Il faut distinguer deux types d'évaluation : l'évaluation de la récupération mesure si les documents récupérés contiennent des informations pertinentes, et en quelle quantité. Cette évaluation intervient après la section Reranking du pipeline (figure I.1). Elle s'effectue en confrontant les documents récupérés aux annotations des experts, à différents seuils k (exprimés en pourcentages du nombre total de chunks).

Ensuite, on évalue le résumé, pour vérifier s'il contient bien les informations présentes dans les documents récupérés, s'il est conforme à la structure attendue, et s'il ressemble suffisamment au résumé de référence. Cette évaluation intervient à la toute fin du pipeline, après la génération.

Nous ne discuterons ici que de l'évaluation de la récupération, des métriques associées, et des métriques associées au reranking des documents.

4.1 Précision, Rappel et F1-score à k

Soit RET_k l'ensemble des k premiers chunks retournés par le modèle, et REL l'ensemble des chunks jugés pertinents dans la "vérité terrain" (Ground Truth).

- **Precision@k (Précision)** : Proportion de documents pertinents parmi les documents retournés.

$$P@k = \frac{|RET_k \cap REL|}{|RET_k|}$$

- **Recall@k (Rappel)** : Proportion de documents pertinents retournés par rapport à l'ensemble total des documents pertinents existants dans le dossier.

$$R@k = \frac{|RET_k \cap REL|}{|REL|}$$

- **F1-score@k** : Moyenne harmonique de la précision et du rappel.

$$F_1@k = 2 \times \frac{P@k \times R@k}{P@k + R@k}$$

4.2 NDCG@k (Normalized Discounted Cumulative Gain)

Le NDCG [11, 12] est un score qui mesure la qualité du classement de la récupération. Il est crucial pour mesurer la performance du reranker car il pénalise les systèmes qui placent les bons résultats en bas de liste. Il prend en compte l'ordre de récupération. Tout d'abord, on définit le DCG (Discounted Cumulative Gain) où $rel_i \in \{0, 1\}$ représente la pertinence du document au rang i :

$$DCG@k = \sum_{i=1}^k \frac{rel_i}{\log_2(i+1)}$$

Le *Ideal DCG* (IDCG) représente le score DCG maximal que l'on pourrait obtenir si tous les documents pertinents étaient classés en tête :

$$IDCG@k = \sum_{i=1}^{\min(k, |REL|)} \frac{1}{\log_2(i+1)}$$

Enfin, le NDCG est obtenu par normalisation :

$$NDCG@k = \frac{DCG@k}{IDCG@k} \in [0, 1]$$

La fonction `evaluate_retrieval` ([Annexe A](#)) calcule l'intégralité de ces métriques et les log dans un fichier CSV.

II Présentation du jeu de données

Les données utilisées dans ce projet proviennent principalement de dossiers patients numérisés. Ces dossiers prennent la forme de fichiers texte souvent très longs, contenant une grande quantité d'informations.

Plusieurs difficultés ont été identifiées. Tout d'abord, le volume de données s'accompagne d'un important bruit informationnel : une grande partie des informations présentes n'est pas directement utile pour l'analyse, ce qui complique l'extraction des éléments pertinents. De plus, les informations importantes sont souvent dispersées dans les documents, sans organisation claire, ce qui rend leur exploitation plus difficile.

Les textes contiennent également de nombreux termes médicaux techniques, parfois complexes, qui rendent leur compréhension et leur traitement automatique délicats. À cela s'ajoutent des incohérences temporelles : certaines informations peuvent être contradictoires, par exemple lorsqu'une maladie est indiquée comme présente à un instant donné puis absente à un autre.

Une difficulté majeure réside dans la faible quantité de données annotées. En effet, seuls 15 dossiers patients ont été annotés, ce qui reste insuffisant pour réaliser des tests robustes ou entraîner efficacement des modèles.

Les données annotées sont structurées par sections correspondant aux différentes catégories d'informations médicales présentes dans un dossier patient. Par exemple, pour un document donné, on retrouve des rubriques telles que le mode de vie, les antécédents ou encore les données biométriques. Pour chacune de ces sections, les informations jugées pertinentes ont été identifiées et distinguées des informations non pertinentes. Cette organisation permet de mieux comprendre où se situent les informations importantes dans le texte et facilite leur exploitation.

En complément, des résumés rédigés par des médecins sont également disponibles. Il s'agit de résumés types, utilisés comme référence afin d'évaluer la qualité des résumés générés automatiquement.

Par ailleurs, les données sont majoritairement non structurées, car issues de texte libre sans format standardisé. Un travail de structuration est donc nécessaire avant toute analyse. On observe également un déséquilibre dans les données : certaines pathologies, parfois importantes, apparaissent très rarement, tandis que d'autres, moins critiques, sont beaucoup plus représentées. Cela peut introduire des biais dans les résultats.

Enfin, un même concept médical peut être exprimé de différentes manières (abréviations, synonymes, formulations variées), ce qui complique encore davantage l'analyse automatique des textes.

Dans l'ensemble, ces caractéristiques rendent le traitement des données particulièrement complexe et nécessitent la mise en place de méthodes adaptées afin d'obtenir des résultats fiables.

III Retriever : Modèles, Expérimentations, Résultats

1 Description globale du pipeline

Le système mis en place est un pipeline en cascade composé de cinq grandes étapes :

1. **Indexation et Chunking** : Le corpus est composé de plusieurs documents en format texte, qui correspond à des comptes rendus d'opérations et de consultations. L'ensemble du corpus est séparé en chunks de taille fixe (en général 512 caractères), afin de travailler avec des sections de la même longueur.
2. **Embedding (Plongement)** : On transforme les chunks et la requête en vecteurs, en utilisant les embeddings des différents modèles.
3. **Retrieval (Récupération de base)** : On effectue un calcul de similarité entre la requête et les chunks pour chaque modèle, pour déterminer les chunks qui correspondent le mieux à la requête.
4. **Fusion Hybride** : On combine les résultats de chaque modèle, grâce à plusieurs méthodes détaillées plus loin.
5. **Reranking** : À l'aide d'un modèle plus lourd, on reclasse les chunks récupérés pour placer les meilleurs candidats à l'avant.

Le processus s'exécute itérativement pour chaque patient. Le nombre de résultats retournés (K) par l'étape de retrieval est dynamique, calculé en pourcentage de la taille totale du dossier du patient (de 5% à 30% des chunks totaux). Dans les graphes suivants, ces seuils sont exprimés par un niveau de 0 à 5 (par exemple, $R@0$ correspond au recall avec 5% des chunks, et $R@5$ au recall avec 30% des chunks).

Dans le but d'assurer un rappel (recall) maximal avant la phase de filtrage, la pipeline implémente un panel varié de modèles, chacun spécialisé dans un type de représentation de l'information.

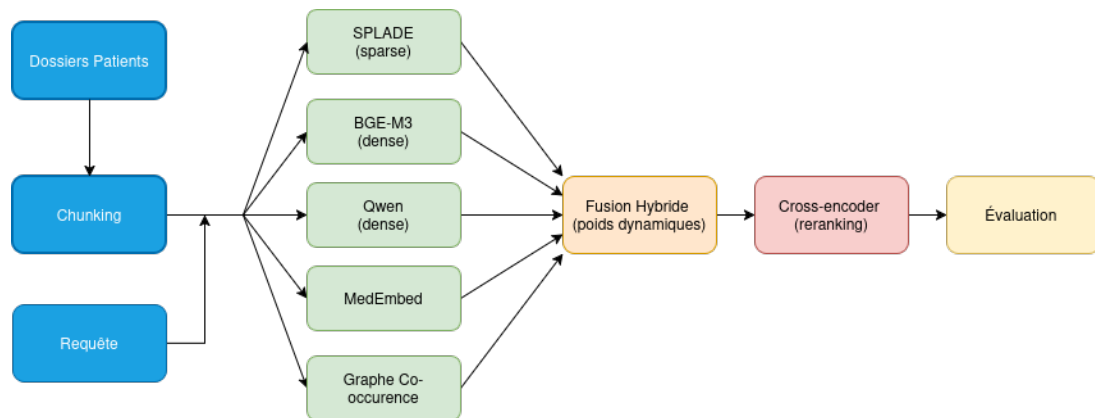


FIGURE III.1 – Architecture du pipeline de récupération et de reranking

2 Retrievers testés

2.1 Configuration Expérimentale et Hyperparamètres

Pour garantir la reproductibilité de nos expériences, les paramètres clés de notre pipeline d'évaluation sont définis comme suit :

- **Découpage des documents (Chunking)** : Taille des segments (*chunk_size*) fixée à 512 tokens, avec un chevauchement (*overlap*) de 0.
- **Métriques d'évaluation** : Précision, Rappel, F1-Score et nDCG (Normalized Discounted Cumulative Gain), mesurés à différents seuils k .
- **Seuils k (Top- k)** : Évalués dynamiquement en fonction de la taille du corpus du patient : $k \in \{5\%, 10\%, 15\%, 20\%, 25\%, 30\%\}$ du nombre total de chunks.
- **Cross-Encoder (Reranking)** : Modèle utilisé BAAI/bge-reranker-v2-m3 [13].

2.2 SPLADE (Sparse Lexical and Expansion Model)

Principe théorique

SPLADE [14, 15] est un modèle de représentation Sparse basé sur l'architecture BERT. Contrairement aux modèles denses qui compressent l'information dans un vecteur de taille fixe et dense, SPLADE génère des vecteurs de la taille du vocabulaire entier du modèle linguistique (souvent plus de 30 000 dimensions), où la majorité des valeurs sont à zéro. L'avantage majeur est qu'il effectue une *expansion lexicale* : il active des dimensions pour des mots qui ne sont pas présents dans le texte mais qui lui sont sémantiquement liés (synonymes).

Formulation mathématique

Soit un token t_i dans un document. Le logit final pour un mot du vocabulaire j est calculé par le Masked Language Model (MLM). Le poids w_j du mot j dans la représentation du document est obtenu via un pooling de type *Max* sur l'ensemble des tokens du document :

$$w_j = \max_{i \in \text{doc}} \log(1 + \text{ReLU}(h_i^T E_j + b_j))$$

où h_i est la représentation contextuelle du token, et E_j, b_j sont l'embedding et le biais du mot j . Le score final de similarité entre une requête Q et un document D est simplement le produit scalaire des deux vecteurs clairsemés : $\text{Score}(Q, D) = \sum_j w_{j,Q} \times w_{j,D}$.

2.3 SentenceTransformers : Modèles Denses (Bi-Encoders)

Contrairement à SPLADE, les modèles denses plongent les requêtes et les documents dans un espace vectoriel continu de dimension réduite (ex : 768 ou 1024).

Principe théorique et Mathématique

Le principe du Bi-Encoder est de posséder deux flux de traitement parallèles (ou un seul appelé deux fois) générant un vecteur U pour la requête et V pour le chunk. La distance entre ces deux vecteurs dans l'espace latent définit leur similarité. La mesure utilisée dans notre pipeline est la similarité cosinus (ou produit scalaire sur vecteurs normalisés) :

$$\text{sim}(U, V) = \cos(\theta) = \frac{U \cdot V}{\|U\| \|V\|}$$

Modèles implémentés

- **multilingual-e5-large** [16] : Optimisé pour le multilinguisme, il nécessite l'ajout de préfixes spécifiques `query:` ou `passage:` avant l'inférence.
- **BGE-M3 (BAAI)** [13] : Un modèle d'état de l'art supportant le multilinguisme (M), plusieurs longueurs de texte, et plusieurs tâches de granularité différente.
- **Qwen Embeddings (0.6B et 8B)** [17] : Modèles basés sur l'architecture Qwen (Alibaba). La version 8B est particulièrement massive, permettant une compréhension fine de la sémantique complexe inhérente aux dossiers médicaux.
- **MedEmbed** [18] : Modèle spécialisé affiné (fine-tuned) exclusivement sur des corpus médicaux. Il est supposé offrir de meilleures performances sur la terminologie clinique spécifique (reconnaissance de molécules, pathologies).

Dans le script, la méthode `model.encode(..., convert_to_numpy=True, normalize_embeddings=True)` est employée, suivie par `np.dot()` ou `util.cos_sim()` pour le calcul rapide des similarités.

2.4 Méthode Graph-based (Centralité avec NetworkX)

Principe théorique

Pour s'affranchir des réseaux de neurones, une méthode reposant sur la théorie des graphes [19] est implémentée. Un graphe de co-occurrence est construit à partir du chunk : chaque nœud est un mot de plus de 5 caractères, et une arête est ajoutée (avec poids croissant) si deux mots apparaissent dans le même chunk.

Formulation mathématique

L'importance d'un mot au sein du texte est mesurée par sa *Degree Centrality* (Centralité de degré) $C_D(v)$. Pour un graphe $G = (V, E)$ avec $|V|$ nœuds :

$$C_D(v) = \frac{\deg(v)}{|V| - 1}$$

Le score d'un chunk pour une requête donnée est alors défini comme la somme des centralités des mots du chunk, à laquelle s'ajoute un bonus constant (+0.5) pour chaque présence explicite d'un terme de la requête.

Résultats

La figure III.2 compare les résultats sur chacune des requêtes. On constate que les modèles n'ont pas tous les mêmes points forts : par exemple, MedEmbed a un recall deux fois supérieur à tous les autres modèles sur la section *biométrie*, mais une faiblesse sur la section *traitements*. La section *allergies* représente un point fort de l'ensemble des modèles. Le modèle hybride, présenté juste après, permet de combiner les forces de chacun des modèles et offre le meilleur recall sur la totalité des requêtes.

Cependant, les modèles individuels ont des résultats inutilisables en pratique : un recall à 30% signifie que seulement 30% des chunks annotés comme pertinents pour cette section ont été récupérés par le modèle ; on a donc 70% des chunks contenant au moins une information pertinente qui ont été laissés de côté. Le besoin d'un modèle plus performant est donc très clair.

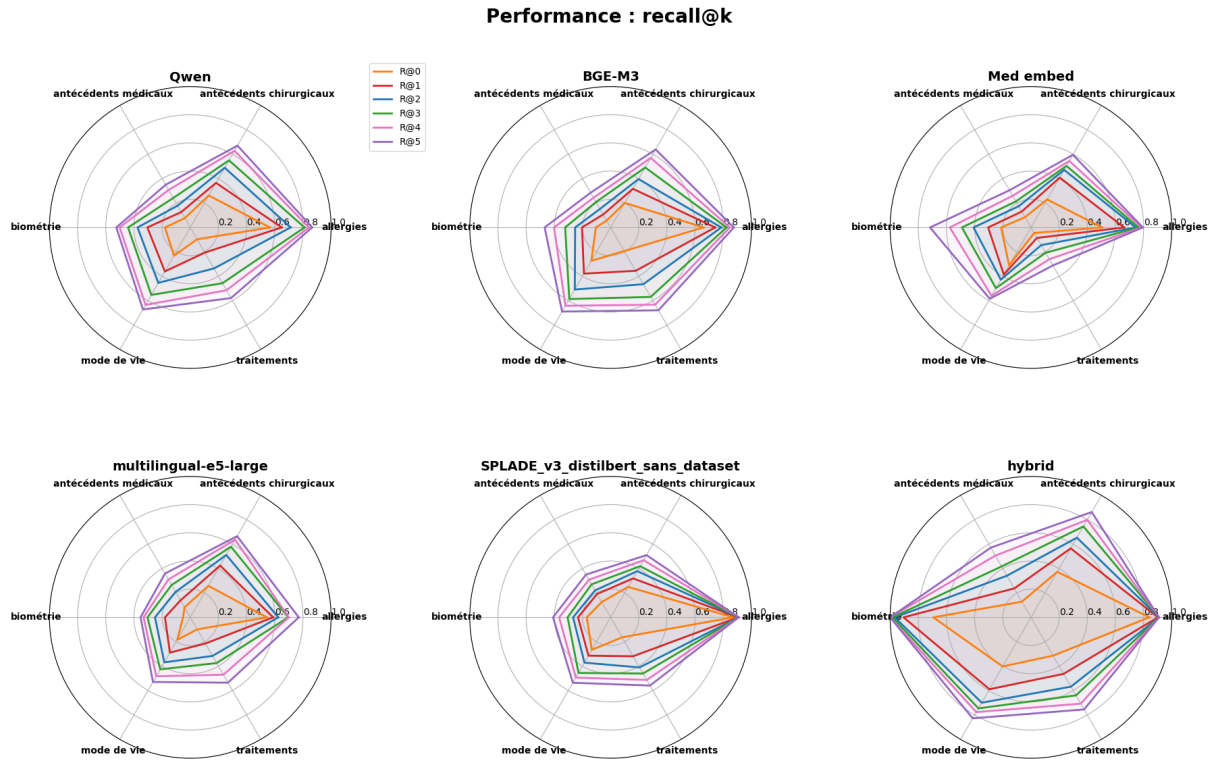


FIGURE III.2 – Comparaison des modèles via le recall à 5, 10, 15, 20 et 30% du total de chunks

3 Premier axe d'amélioration : la fusion hybride

3.1 Définition et Mathématiques

Les modèles denses et sparse ont des forces complémentaires. La fusion hybride vise à combiner les listes de résultats de chaque retriever en un score unifié. Avant fusion, les scores de chaque modèle M_i sont normalisés par un Min-Max scaling pour ramener les distributions entre $[0, 1]$:

$$s'_{c, M_i} = \frac{sc, M_i - s_{min, M_i}}{s_{max, M_i} - s_{min, M_i}}$$

Le score hybride d'un chunk c pour une requête q est la somme pondérée des scores normalisés :

$$S_{hybride}(c, q) = \sum_{i=1}^{N_{model}} W_{i, q} \times s'_{c, M_i}$$

3.2 Poids optimisés selon les requêtes

Une innovation clé de cette pipeline réside dans l'utilisation de **poids optimisés** ($W_{i, q}$) en fonction de la catégorie de la requête. Par exemple :

- **Requête "biométrie"** : Fait souvent appel à des nombres exacts (poids, tension). Le poids accordé au modèle lexical (SPLADE) et à sa version DSPy [20] est très fort (0.8).
- **Requête "mode de vie"** : Concept flou (sédentarité, tabagisme, stress). Le poids est basculé vers les modèles denses massifs capables de comprendre des abstractions sémantiques (Qwen obtient un poids de 0.5).

3.3 Cross-Encoder (Le Reranking)

Différence Bi-Encoder vs Cross-Encoder

Lors de la première étape de recherche (Bi-Encoder), les requêtes et les documents sont traités séparément, ce qui est rapide mais empêche le modèle d’observer l’interaction entre les mots de la requête et ceux du document. Un Cross-Encoder concatène la requête et le document en entrée d’un seul réseau de neurones Transformer :

$$\text{Input} = [\text{CLS}] \text{ Requête } [\text{SEP}] \text{ Document } [\text{SEP}]$$

L’attention croisée (Self-Attention) s’opère sur l’ensemble de la séquence, permettant une évaluation de similarité contextuelle beaucoup plus précise.

Impact attendu et Utilisation

Le Cross-Encoder étant très lourd computationnellement, il n’est appliqué qu’en phase de *reranking* sur une portion restreinte des données (ici, le Top 30% des candidats issus de la fusion hybride). Le modèle BAAI/bge-reranker-v2-m3 [13] génère un `cross_score` qui écrase le score hybride pour ordonner la liste finale. L’impact attendu est une hausse significative de la précision au sommet de la liste (Precision@5, Precision@10).

3.4 Résultats

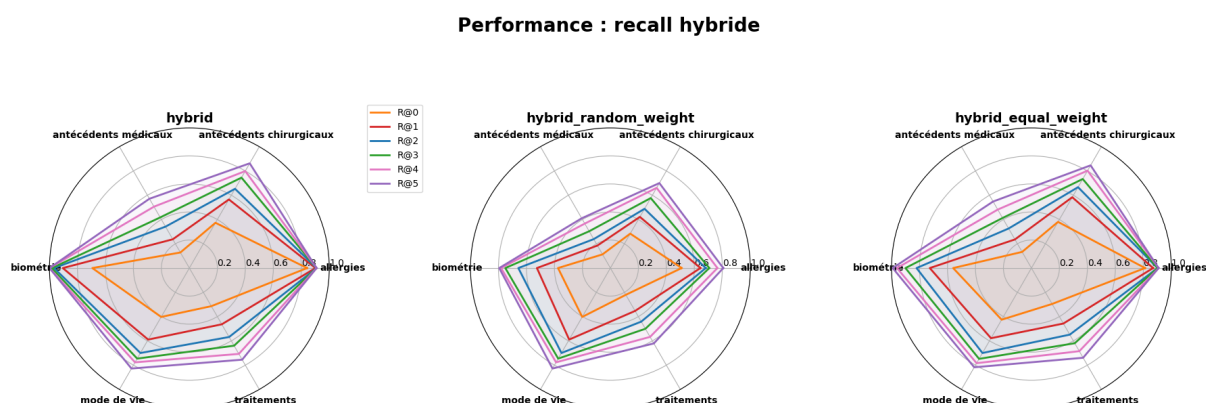


FIGURE III.3 – Comparaison via le recall du modèle hybride avec poids optimisés par requête, poids uniformes et poids aléatoires

On observe sur la figure III.3 des résultats nettement supérieurs avec les modèles à poids uniformes et optimaux, avec le recall et la précision, que le modèle à poids aléatoires. De plus, le modèle à poids optimaux obtient partout des résultats équivalents ou meilleurs que le modèle à poids uniformes.

On remarque également que le modèle hybride est nettement meilleur que les modèles dense ou sparse individuels (figure III.2), avec notamment un point fort sur la biométrie, qui est une faiblesse de la majorité des modèles. Il offre un recall supérieur à 90% dans quatre catégories sur six, ce qui en fait un candidat crédible pour le pipeline final.

On observe aussi sur la figure III.4 les résultats attendus quant à la précision du modèle hybride par rapport aux modèles dense ou sparse : Qwen présente une précision inférieure à 40% pour toutes les requêtes même avec 5% des chunks du dossier récupérés, mais garde un score

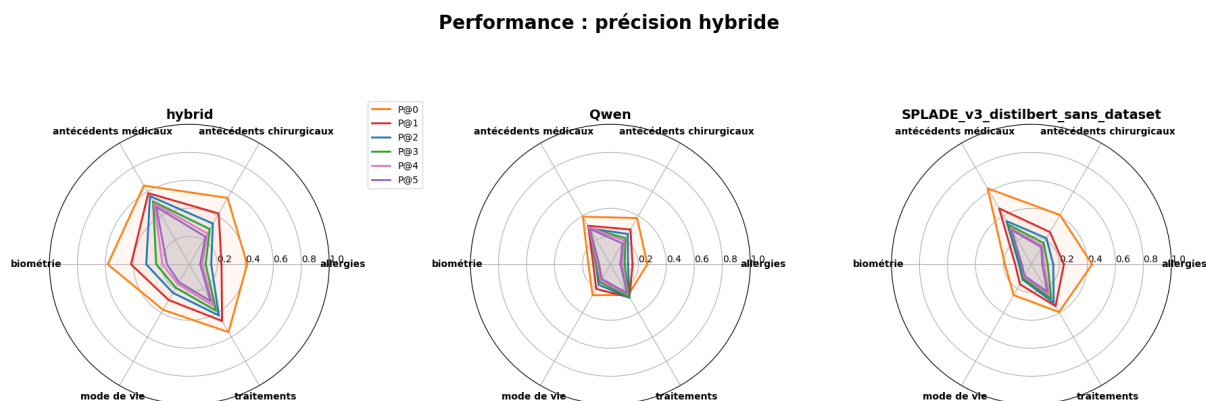


FIGURE III.4 – Comparaison via la précision du modèle hybride à poids optimisés avec un modèle dense et un modèle sparse

constant quand le nombre de chunks récupérés augmente. Cela montre que le modèle est capable de trouver les documents pertinents, mais a du mal à les séparer du reste.

SPLADE a une bonne précision avec 5% des chunks récupérés (entre 20% et 40% selon les requêtes, 60% sur les antécédents médicaux), mais le score diminue rapidement à mesure que le nombre de chunks augmente. Il a donc du mal à repérer les chunks moins "évidents".

Le reranker du modèle hybride permet de reclasser les chunks récupérés par chacun des modèles, et on constate que ce système est efficace, car la précision avec 5% des chunks du dossier est supérieure à 40% pour toutes les requêtes.

Cependant, certaines requêtes gardent des résultats relativement faibles : la section *antécédents médicaux* atteint difficilement 60% de recall, ce qui est loin d'être suffisant pour garantir la fiabilité du résumé final. Nous allons donc étudier un second axe d'amélioration pour essayer de corriger cette faiblesse, et améliorer les résultats globaux.

4 Second axe d'amélioration : optimisation des requêtes

4.1 Query expansion par datasets

Dans le cadre de l'optimisation de notre système de recherche d'information, notre attention s'est d'abord portée sur deux axes spécifiques : les antécédents médicaux et le mode de vie des patients. Ces catégories présentaient encore un recall (taux de rappel) faible et nécessitaient d'être améliorées.

Pour y remédier, notre méthodologie s'est articulée en deux étapes. Premièrement, nous avons procédé à un fine-tuning (réentraînement) du modèle SPLADE afin de spécialiser ses capacités d'expansion lexicale sur notre corpus. Deuxièmement, nous avons cherché à enrichir les représentations vectorielles (embeddings) en y intégrant du vocabulaire médical ciblé. Pour ce faire, nous avons exploité des bases de données fournies par l'État (comme les nomenclatures CIM [21] et MeSH [22]) que nous avons retravaillées et structurées au format JSON.

Bien que les évaluations montrent une amélioration des performances sur ces axes spécifiques, cette approche se heurte à une réalité terrain qui en limite le plein potentiel. Le problème fondamental réside dans la nature même des connaissances injectées : les dictionnaires CIM et MeSH utilisent une terminologie médicale stricte, standardisée et académique. Or, dans la pratique clinique quotidienne, les médecins rédigent leurs notes de manière beaucoup plus informelle et

rapide. Ils utilisent massivement des abréviations, du jargon clinique, des acronymes non standards et des raccourcis syntaxiques.

Par conséquent, l'écart de langage entre la rigueur sémantique des dictionnaires et la donnée textuelle brute produite par les praticiens crée un "bruit" résiduel. L'injection de ces connaissances peine donc à s'aligner avec le texte réel, ce qui explique que la pertinence des prédictions ne soit pas toujours optimale malgré l'apport indéniable du dataset.

C'est pour ces raisons que nous n'arrivons pas à récupérer l'ensemble des extraits pertinents (chunks) et à obtenir un recall plus élevé. Une méthode beaucoup plus pertinente serait d'améliorer les embeddings en les adaptant directement au jargon médical "terrain". Cela nous permettrait d'être beaucoup plus pertinents, car nous enlèverions une grande partie de ce bruit résiduel, garantissant ainsi un bien meilleur recall.

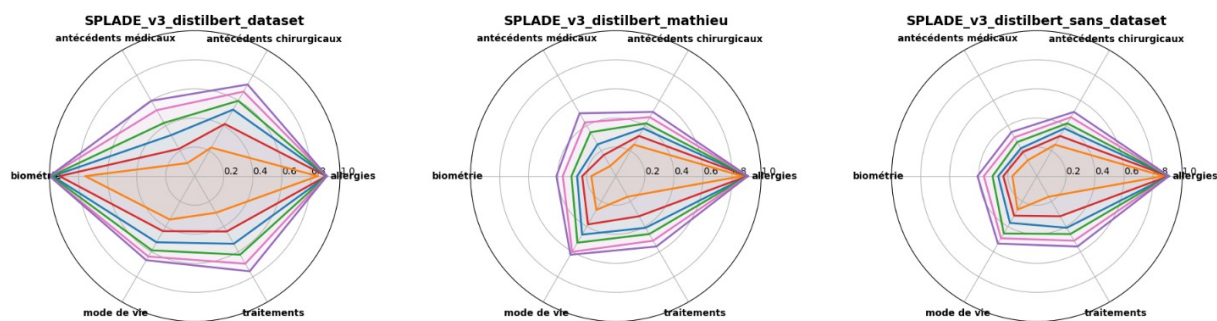


FIGURE III.5 – Comparaison de la qualité de récupération selon le dataset utilisé

C'est d'ailleurs très exactement ce que prouvent nos derniers résultats (figure III.5). En observant la comparaison des trois graphiques, l'évolution est frappante entre le modèle de base de droite (sans_dataset), le modèle central enrichi avec les dictionnaires rigides (mathieu) et notre modèle de gauche, directement adapté aux vraies données et au jargon du terrain (dataset).

Le graphique de gauche montre une progression spectaculaire. En se nourrissant de la véritable terminologie utilisée par les médecins au quotidien, le modèle déploie littéralement ses performances. On observe une poussée impressionnante sur l'axe de la "biométrie" (qui atteint désormais le score maximum), mais aussi un élargissement net et global sur toutes les autres catégories, y compris les "antécédents médicaux" et le "mode de vie". Le polygone est beaucoup plus vaste et équilibré. Cela confirme totalement notre hypothèse : en s'adaptant à la réalité brouillonne des notes cliniques plutôt qu'à la rigidité d'un dictionnaire, on supprime le fameux bruit résiduel. Le moteur comprend enfin le vrai langage des praticiens et parvient à retrouver massivement les bons extraits de texte, garantissant ce recall largement supérieur.

4.2 Query expansion & rewriting via DSPy

Afin d'améliorer les performances de la phase de retrieval, nous avons exploré l'optimisation automatique des prompts via le framework DSPy [20]. Plutôt que de rédiger manuellement les instructions envoyées au modèle de langage, DSPy permet de compiler un module en ajustant automatiquement ses prompts pour maximiser une métrique définie — ici le Recall@20, c'est-à-dire la proportion de chunks pertinents retrouvés parmi les 20 premiers résultats du retriever lexical.

Le module optimisé encapsule une unique Signature DSPy chargée de transformer la ques-

tion initiale en une liste dense de mots-clés médicaux adaptés au retriever lexical (TF-IDF / SPLADE). Deux stratégies d'optimisation ont été évaluées.

BootstrapFewShot ([documentation](#)) est la stratégie la plus simple : elle génère automatiquement des exemples few-shot à partir d'un trainset en sélectionnant les traces d'exécution dont le Recall@20 dépasse un seuil défini (`metric_threshold`). Concrètement, pour chacune des six sections du résumé (mode de vie, antécédents médicaux, antécédents chirurgicaux, allergies, traitements, biométrie), un trainset dédié de dix questions thématiques a été constitué. Le module a été compilé section par section, et un garde-fou a été implémenté pour ne retenir la version compilée que si elle surpasse le score baseline en zero-shot.

KNNFewShot ([documentation](#)) adopte une approche dynamique : plutôt qu'un ensemble fixe de démos, il sélectionne à chaque inférence les k exemples sémantiquement les plus proches de la question courante, via un modèle d'embeddings médicaux (MedEmbed-large-v0.1). Le trainset global de 60 questions couvrant les six sections a été vectorisé une fois, et le module sélectionne à chaque appel les 3 voisins les plus proches pour les injecter comme démos dans le prompt. Cette approche ne nécessite pas de phase d'optimisation coûteuse et est particulièrement adaptée à la diversité thématique des sections.

KNNFewShot ne présente pas de résultat significatif, et n'apporte rien par rapport à BootstrapFewShot pour générer un résumé. Cependant, un tel outil peut se révéler utile lors d'une discussion type question-réponse avec le modèle, afin de donner une réponse à une question précise du médecin sur le dossier patient. Les résultats présentés ci-après concernent donc BootstrapFewShot.

Résultats

Nous avons comparé les performances des modèles de base ("Sans DSPy") par rapport aux modèles bénéficiant d'une expansion et d'une contextualisation des requêtes ("Avec DSPy").

Comme illustré dans la Figure [III.6](#), qui présente la métrique de Rappel ($R@k$) moyennée, l'utilisation de DSPy améliore systématiquement les scores de récupération pour l'ensemble des modèles testés.

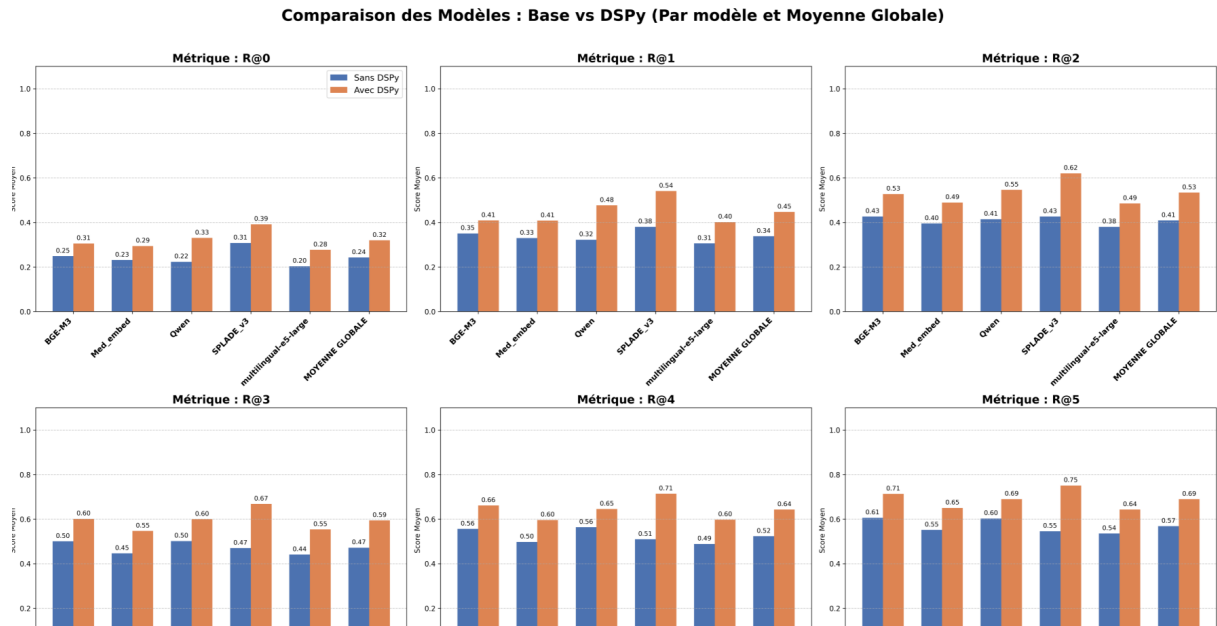


FIGURE III.6 – Comparaison des métriques de Rappel ($R@0$ à $R@5$) par modèle, illustrant l’impact de l’optimisation DSPy par rapport aux requêtes de base.

4.3 HyDE

La méthode HyDE [23] (Hypothetical Document Embeddings) consiste à faire générer par un modèle de langage un document fictif qui répond le mieux possible à la question. Cette méthode a pour objectif de capturer lors de la recherche vectorielle des documents au sens proche du document fictif. Dans notre cas, la création d’un document fictif permet de capter la structure des documents du corpus et éventuellement des termes susceptibles d’être présents dans le corpus.

Les expérimentations avec cette méthode ont été menées pus tôt dans le projet, avec des modèles différents de ceux utilisés à la fin du projet (notamment BM25 et sentence-camembert), et les résultats obtenus sont difficiles à comparer avec ceux présentés ici. Cependant, les résultats obtenus avec DSPy surpassent largement ceux obtenus avec HyDE. Comme les deux approches offrent une expansion de requête similaire, nous avons choisi de continuer avec DSPy. Les résultats obtenus avec HyDE sont présentés en [Annexe C](#)

5 Analyse des Résultats et Justification des Choix Techniques

5.1 Résultats globaux

La recherche d'informations dans un dossier patient n'est pas uniforme ; certaines requêtes ciblent des données structurées (biométrie) et d'autres des descriptions narratives (mode de vie). La Figure III.7 présente les performances au format radar selon les différentes catégories d'annotations.

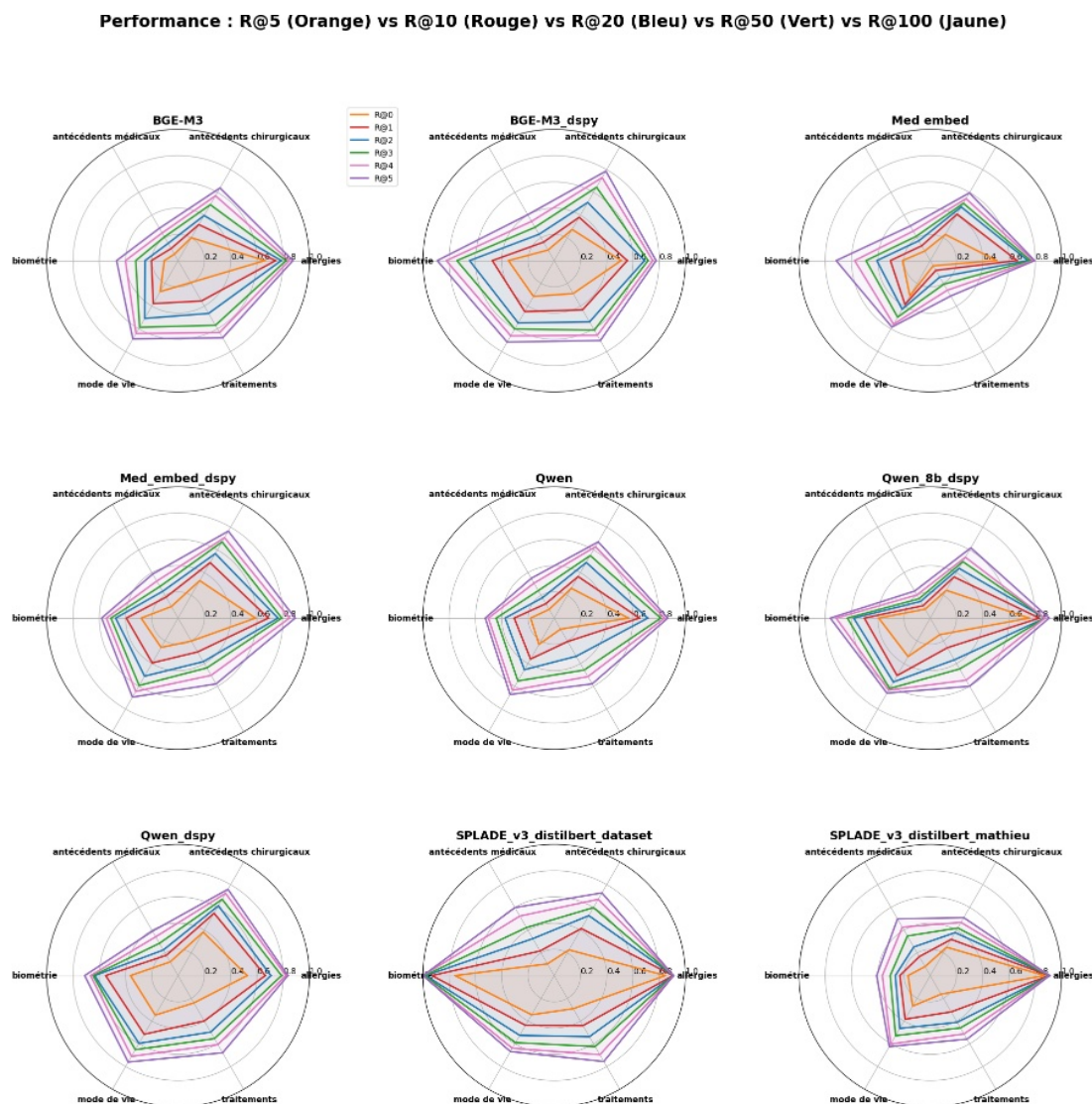


FIGURE III.7 – Graphiques en radar illustrant le Rappel (R@5 à R@100) pour chaque modèle en fonction des catégories d'informations cliniques (antécédents, traitements, biométrie, etc.).

TABLE III.1 – Comparaison des performances globales des différents Retrievers (Valeurs moyennes sur l'ensemble du dataset).

Modèle / Configuration	R@0	R@1	R@2	R@3	R@4	R@5
<i>Modèles de Base (Denses & Sparse)</i>						
BGE-M3	0.248	0.350	0.427	0.500	0.556	0.606
Med Embed	0.231	0.329	0.395	0.446	0.498	0.551
multilingual-e5-large	0.203	0.306	0.380	0.441	0.488	0.535
Qwen (0.6B)	0.236	0.335	0.424	0.509	0.571	0.610
SPLADE v3	0.308	0.380	0.426	0.470	0.510	0.546
<i>Modèles optimisés (avec DSPy)</i>						
BGE-M3 + DSPy	0.305	0.409	0.527	0.601	0.661	0.713
Med Embed + DSPy	0.294	0.408	0.489	0.547	0.596	0.650
multilingual-e5-large + DSPy	0.277	0.401	0.485	0.554	0.597	0.643
Qwen (0.6B) + DSPy	0.330	0.476	0.546	0.600	0.645	0.690
Qwen (8B) + DSPy	0.326	0.429	0.495	0.540	0.585	0.630
SPLADE v3 + DSPy	0.391	0.540	0.620	0.668	0.714	0.751
<i>Approches de Fusion (Hybrides & Reranking)</i>						
Hybride (Poids Égaux)	0.434	0.575	0.661	0.719	0.770	0.809
Hybride (Poids Aléatoires)	0.317	0.453	0.535	0.597	0.653	0.693
Hybride (Poids optimaux)	0.459	0.610	0.690	0.740	0.783	0.821
Hybride + Cross-Encoder	0.298	0.458	0.554	0.668	0.750	0.821

Note : L'approche hybride combine les scores normalisés de SPLADE, MedEmbed, multilingual-e5, BGE-M3, Qwen .

L'analyse des résultats quantitatifs présentés dans le tableau III.1 met en évidence plusieurs dynamiques fondamentales concernant l'architecture de notre pipeline de recherche d'information en domaine médical.

5.2 Analyse et interprétation

Les résultats montrent un gain net après expansion de requêtes avec DSPy. Dans ce cadre, l'ajout de DSPy améliore le rappel des modèles testés : SPLADE v3 passe de 0.308 à 0.391 sur le score R@0, soit une hausse de 27%, tandis que BGE-M3 passe de 0.606 à 0.713 sur le score R@5. L'expansion de requêtes améliore donc la correspondance entre des requêtes courtes et le contenu des dossiers cliniques.

Les résultats de base montrent également une différence entre les modèles denses et sparse selon le seuil considéré. Sans DSPy, SPLADE v3 obtient un score R@0 de 0.308, contre 0.248 pour BGE-M3. En revanche, à R@5, BGE-M3 atteint 0.606, contre 0.546 pour SPLADE. Les modèles sparse obtiennent donc de meilleurs résultats sur les premiers rangs, tandis que les modèles denses obtiennent de meilleurs scores lorsque davantage de documents sont pris en compte.

On observe que la combinaison des modèles améliore les performances par rapport aux approches prises séparément. Une fusion avec des poids égaux atteint déjà un score R@5 de 0.809, supérieur à celui des modèles individuels. La stratégie à poids optimaux obtient les meilleurs

résultats observés, avec un score $R@0$ de 0.459 et un score $R@5$ de 0.821. Ces résultats montrent l'intérêt d'ajuster les poids de fusion selon la catégorie de la requête.

Cependant, l'ajout d'un Cross-Encoder en fin de pipeline ne conduit pas ici à une amélioration sur les premiers rangs. Le score $R@5$ reste à 0.821, mais le score $R@0$ passe de 0.459 avec l'hybride dynamique seul à 0.298 avec l'ajout du Cross-Encoder. Dans cette configuration, le reranking ne dégrade pas le rappel global à $R@5$, mais il modifie l'ordre des premiers résultats de manière défavorable pour $R@0$. Une explication possible est que le modèle `BAAI/bge-reranker-v2-m3`, entraîné sur des données généralistes, soit moins adapté au vocabulaire médical français et aux formulations propres aux dossiers cliniques.

5.3 Évaluation du compromis Temps vs Performance (Latence)

Le déploiement d'un système de recherche d'information (Retrieval) en milieu hospitalier impose des contraintes de latence strictes. Afin d'évaluer la viabilité de notre pipeline en conditions réelles, nous avons modélisé le compromis entre le temps de récupération moyen (en secondes) et l'efficacité (métrique $R@5$).

La Figure III.8 illustre la distribution des modèles selon ces axes. Pour quantifier cette dynamique, nous présentons la confrontation directe entre la latence algorithmique et la performance pour les modèles individuels dans la Table III.2, et l'évaluation du surcoût des architectures de fusion dans la Table III.3.

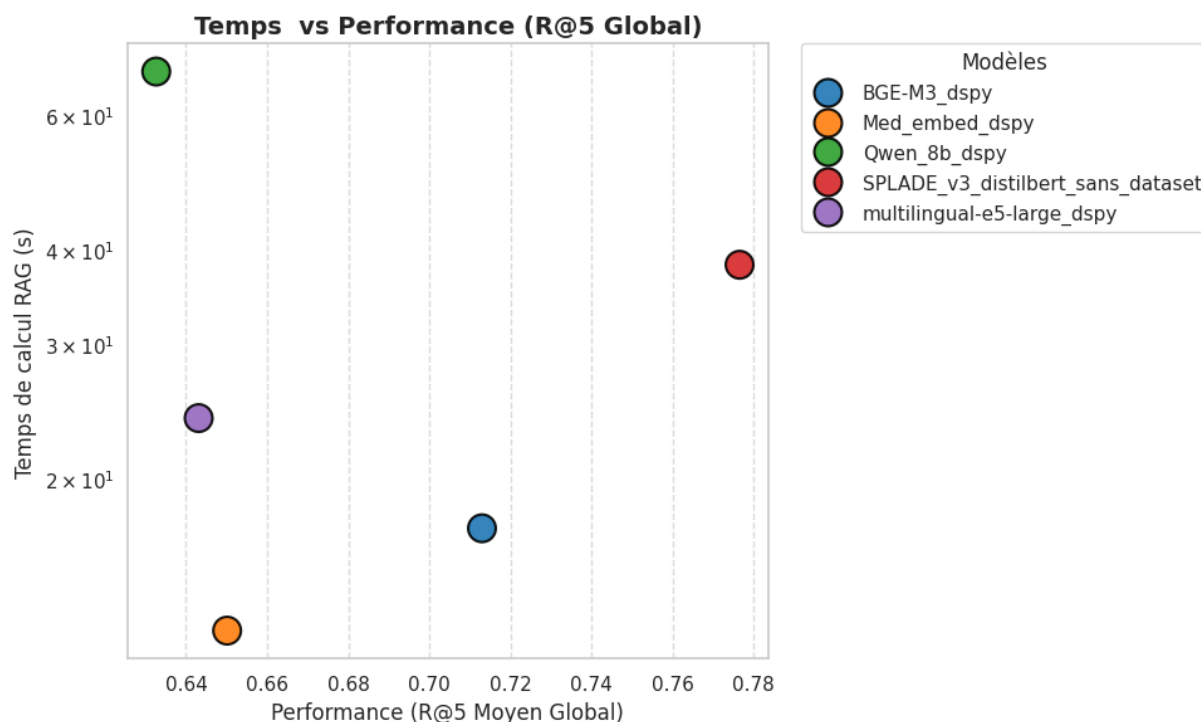


FIGURE III.8 – Nuages de points illustrant le compromis entre le temps de calcul (axe Y, échelle logarithmique) et la performance $R@5$ (axe X).

TABLE III.2 – Compromis : Temps (Allergies/301) vs Performance (R@5 Global) pour la sélection de modèles

Modèle	R@5 Global	Temps RAG (s)
Med_embed_dspy	0.650	12.6170
BGE-M3_dspy	0.713	17.1929
multilingual-e5-large_dspy	0.643	23.9926
SPLADE_v3_distilbert_sans_dataset	0.777	38.1459
Qwen_8b_dspy	0.633	68.4173

TABLE III.3 – Évaluation du surcoût computationnel des architectures de fusion et de Reranking.

Modèle / Configuration	R@5 Moyen	Temps Moyen (s)
Hybride (Poids optimaux)	0.821	0.0009
Hybride (Poids Aléatoires)	0.693	0.0009
Hybride + Cross-Encoder	0.821	0.8182
Hybride (Aléatoire) + Cross-Encoder	0.693	0.8532

Analyse critique des architectures et recommandations

La mise en perspective de ces deux tableaux met en avant plusieurs dynamiques fondamentales pour le choix de l'architecture cible :

La dichotomie Dense/Sparse et le coût de l'architecture LLM. Parmi les modèles autonomes (Table III.2), `Med_embed_dspy` s'avère le plus rapide (12.61 s) mais offre un rappel limité (0.650). Le modèle `BGE-M3_dspy` représente un excellent compromis opérationnel : il maintient un temps d'exécution raisonnable (17.19 s) tout en augmentant significativement l'efficacité (0.713). À l'inverse, si `SPLADE` garantit la meilleure précision absolue individuelle (R@5 = 0.777), son approche Sparse se paie au prix fort avec une latence de 38.14 s, soit plus du double de `BGE-M3`. Enfin, l'utilisation de modèles fondés sur des LLMs massifs (comme `Qwen_8b`) se révèle totalement inadaptée à cette étape primaire de recherche : leur lourdeur architecturale fait exploser la latence (68.41 s) tout en affichant les pires performances de la sélection (0.633).

Le paradoxe computationnel de la Fusion Hybride. L'approche hybride à poids optimaux (Table III.3) domine le classement en termes de performance absolue avec un R@5 de 0.821 et une latence algorithmique affichée de seulement 0.0009 s. Il est cependant crucial de contextualiser cette latence quasi nulle : elle représente uniquement l'opération mathématique d'agrégation des scores. Dans un environnement de production réel, ce gain de performance nécessite de maintenir plusieurs modèles d'embeddings en mémoire et d'orchestrer leurs requêtes en parallèle, imposant une infrastructure complexe. La comparaison avec l'hybride "Aléatoire" (0.693) confirme d'ailleurs que cette surperformance provient exclusivement de la pertinence des poids optimisés, et non de la simple accumulation des modèles.

Le surcoût du Cross-Encoder. L'étape de Reranking par Cross-Encoder soulève un enjeu d'efficacité notable. Son introduction ajoute un surcoût d'environ 0.81 s à l'étape d'agrégation. Bien que cette valeur puisse sembler marginale face aux dizaines de secondes du processus global

(RAG complet), cette opération lourde de calcul d'attention croisée n'apporte aucune amélioration sur le rappel global à $k = 5$, qui stagne à 0.821 pour les poids optimaux. Son utilisation s'avère donc redondante pour cette métrique spécifique, ajoutant une charge serveur sans bénéfice direct sur le top-5.

Conclusion pour le déploiement : Pour un passage à l'échelle hospitalière, la stratégie la plus performante consiste à privilégier l'approche **Hybride à poids optimaux sans Cross-Encoder**, à la stricte condition de disposer d'une infrastructure permettant l'inférence parallèle fluide des modèles sous-jacents. À défaut, le modèle **BGE-M3_dspy** s'impose de loin comme l'alternative "mono-RAG" la plus rationnelle, offrant le meilleur équilibre pratique entre temps de réponse (17 s) et pertinence documentaire (0.713).

IV Conclusion et perspectives

1 Conclusion du projet

Au cours de ce projet, nous avons cherché à répondre à un enjeu concret du domaine médical : faciliter l'accès à l'information pertinente au sein de dossiers patients volumineux et hétérogènes. En nous appuyant sur une architecture de type RAG, nous avons conçu un pipeline complet combinant plusieurs approches de recherche d'information, afin d'optimiser la pertinence des éléments extraits.

Nos expérimentations ont montré que les modèles pris individuellement présentent des limites importantes, notamment en termes de rappel. Ce constat nous a conduits à explorer des stratégies de combinaison, en particulier la fusion hybride, qui s'est révélée être une approche efficace pour tirer parti de la complémentarité entre modèles denses et sparse. L'optimisation dynamique des poids ainsi que l'amélioration des requêtes ont également joué un rôle déterminant dans les performances obtenues.

Cependant, ce travail met également en évidence plusieurs limites. La dépendance à des données peu nombreuses et partiellement annotées restreint la portée des évaluations. De plus, certains composants, comme le reranking par cross-encodeur, introduisent des compromis importants entre performance et coût computationnel. Enfin, la généralisation du système à d'autres contextes cliniques reste une question ouverte.

Au-delà des résultats techniques, ce projet souligne l'importance d'une approche pragmatique, où les choix méthodologiques doivent être guidés à la fois par les performances mesurées et par les contraintes d'usage réelles. Dans un contexte aussi sensible que le domaine médical, la fiabilité, la traçabilité et la compréhension des limites du système sont essentielles.

Les perspectives d'amélioration sont nombreuses, notamment en ce qui concerne l'augmentation du volume de données annotées, l'adaptation des modèles au domaine médical spécifique, et l'évaluation en conditions réelles auprès des cliniciens. À terme, ce type d'outil pourrait contribuer à améliorer la qualité de la prise en charge des patients, à condition de poursuivre les efforts de validation et d'intégration dans les pratiques existantes.

2 Perspectives d'amélioration

La perspective principale serait d'implémenter un algorithme de "Learning to Rank" (L2R) pour entraîner un modèle léger capable de déduire automatiquement les poids optimaux (les W_i) de l'hybridation, en se basant sur une fonction de perte (Loss) optimisant directement le NDCG. Par ailleurs, l'introduction d'un chevauchement dynamique lors du chunking et l'exploitation des LLMs en mode génératif pour auto-corriger les récupérations complèteraient parfaitement cette pipeline d'extraction (retrieval) pour former un système RAG complet de bout en bout.

3 Retour d'expérience

Au terme de ce projet, le bilan global est positif. Nous avons posé des bases solides pour un modèle capable de générer un résumé automatisé, fiable et structuré à partir de dossiers patients, et la création de l'interface, l'évaluation clinique et le déploiement sont en passe de débiter.

Cependant, nous avons dû surmonter plusieurs défis majeurs inhérents au domaine médical et à l'intelligence artificielle. Le premier obstacle fut la nature même de la donnée : les dossiers médicaux se sont révélés particulièrement hétérogènes, non standardisés, et parfois incomplets, ce qui complexifie grandement l'extraction d'informations. Sur le plan technique, nous avons été confrontés au compromis délicat entre la performance du modèle, son coût de calcul et son temps d'exécution. Enfin, le faible nombre de dossiers médicaux préalablement annotés a rendu une vraie validation clinique difficile à mettre en place.

Pour nous permettre d'être efficace tout au long du projet, nous avons mis en place et validé plusieurs bonnes pratiques. La structuration claire de notre pipeline RAG a été un atout majeur pour isoler les problèmes et tester de nouvelles briques. L'approche empirique, basée sur des itérations fréquentes et des tests comparatifs (comme l'évaluation des méthodes de recherche denses, sparse et hybrides), nous a permis d'optimiser nos résultats en continu. Tout ceci n'aurait pas été possible sans un encadrement rigoureux : le maintien de réunions régulières, le suivi précis d'un diagramme de GANTT et l'adoption d'une méthode Agile ont garanti la cohésion et la réactivité de l'équipe tout au long du cycle de développement.

En conclusion, ce projet industriel a été une expérience très enrichissante. Sur le plan technique, il nous a permis de découvrir et de nous spécialiser sur les architectures RAG ainsi que sur l'utilisation des modèles de type LLM, avec des problématiques concrètes.

Au-delà des aspects techniques, ce projet nous a aussi permis de renforcer nos compétences en travail d'équipe et en gestion de projet, en travaillant main dans la main avec une équipe de l'AP-HP.

4 Organisation

Afin de mener efficacement le projet avec l'équipe de l'AP-HP, nous avons adopté une organisation en deux sous-équipes complémentaires, permettant de travailler en parallèle sur les briques clés du système. En complément de cette organisation, nous avons mis en place un diagramme de Gantt afin de structurer et de planifier l'ensemble des tâches du projet.

D'une part, deux membres de l'équipe se sont concentrés sur la conception et l'optimisation des requêtes (prompt engineering). Le but est d'améliorer les requêtes afin d'obtenir un recall plus élevé. Ces tâches ont été planifiées et positionnées dans le Gantt de manière à permettre des itérations régulières, et surtout permettre à la seconde équipe d'intégrer les avancées à leur travail.

D'autre part, les deux autres membres ont travaillé sur la sélection, l'évaluation et la comparaison des modèles de langage. Leurs travaux ont également été intégrés dans le diagramme, avec des phases de tests, d'analyse et de prise de décision clairement identifiées.

Le diagramme de Gantt a aussi servi d'outil de coordination entre les deux sous-équipes, et entre notre groupe et l'équipe de l'AP-HP. Il a permis d'organiser des points de synchronisation à des moments clés et d'assurer la cohérence globale du projet, notamment en alignant les phases d'optimisation des requêtes avec les tests sur les modèles.

Enfin, ce planning nous a aidés à suivre l'avancement du projet, à anticiper d'éventuels

retards et à ajuster la charge de travail si nécessaire. L'intégration finale a ainsi pu être réalisée de manière structurée, en respectant les échéances définies dans le Gantt, pour aboutir à un pipeline RAG cohérent, optimisé et fonctionnel

Bibliographie

- [1] Lydie BEDNARCZYK et al. « Scientific Evidence for Clinical Text Summarization Using Large Language Models : Scoping Review ». In : *Journal of Medical Internet Research* 27.1 (15 mai 2025). Publisher : JMIR Publications Inc., Toronto, Canada, e68998. DOI : [10.2196/68998](https://doi.org/10.2196/68998). URL : <https://www.jmir.org/2025/1/e68998> (visit  le 29/12/2025).
- [2] HUGGINGFACE. *Les tokenizers*. 2021. URL : <https://huggingface.co/learn/llm-course/fr/chapter2/4>.
- [3] Jacob DEVLIN et al. « BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding ». In : *CoRR* abs/1810.04805 (2018). arXiv : [1810.04805](https://arxiv.org/abs/1810.04805). URL : <http://arxiv.org/abs/1810.04805>.
- [4] IBM. *Qu'est-ce qu'un sac de mots ?* Consult  le 5 avril 2026. D c. 2023. URL : <https://www.ibm.com/fr-fr/think/topics/bag-of-words>.
- [5] Karen SPARCK JONES. « A statistical interpretation of term specificity and its application in retrieval ». In : *Document Retrieval Systems*. GBR : Taylor Graham Publishing, 1988, p. 132-142. ISBN : 0947568212.
- [6] Nils REIMERS et Iryna GUREVYCH. « Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks ». In : *CoRR* abs/1908.10084 (2019). arXiv : [1908.10084](https://arxiv.org/abs/1908.10084). URL : <http://arxiv.org/abs/1908.10084>.
- [7] Ashish VASWANI et al. *Attention Is All You Need*. 2017. arXiv : [1706.03762](https://arxiv.org/abs/1706.03762) [cs.CL]. URL : <https://arxiv.org/abs/1706.03762>.
- [8] Alec RADFORD et al. « Language Models are Unsupervised Multitask Learners ». In : 2019. URL : <https://api.semanticscholar.org/CorpusID:160025533>.
- [9] Stephen E. ROBERTSON et Karen SPARCK JONES. « Relevance weighting of search terms ». In : *Document Retrieval Systems*. GBR : Taylor Graham Publishing, 1988, p. 143-160. ISBN : 0947568212.
- [10] Louis MARTIN et al. « CamemBERT : a Tasty French Language Model ». In : *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Sous la dir. de Dan JURAFSKY et al. Online : Association for Computational Linguistics, juill. 2020, p. 7203-7219. DOI : [10.18653/v1/2020.acl-main.645](https://doi.org/10.18653/v1/2020.acl-main.645). URL : <https://aclanthology.org/2020.acl-main.645/>.
- [11] Yining WANG et al. « A Theoretical Analysis of NDCG Type Ranking Measures ». In : *CoRR* abs/1304.6480 (2013). arXiv : [1304.6480](https://arxiv.org/abs/1304.6480). URL : <http://arxiv.org/abs/1304.6480>.
- [12] Kalervo J RVELIN et Jaana KEK L INEN. « Cumulated gain-based evaluation of IR techniques ». In : *ACM Trans. Inf. Syst.* 20.4 (oct. 2002), p. 422-446. ISSN : 1046-8188. DOI : [10.1145/582415.582418](https://doi.org/10.1145/582415.582418). URL : <https://doi.org/10.1145/582415.582418>.
- [13] Jianlv CHEN et al. *M3-Embedding : Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. 2025. arXiv : [2402.03216](https://arxiv.org/abs/2402.03216) [cs.CL]. URL : <https://arxiv.org/abs/2402.03216>.

- [14] Thibault FORMAL, Benjamin PIWOWARSKI et Stéphane CLINCHANT. *SPLADE : Sparse Lexical and Expansion Model for First Stage Ranking*. 2021. arXiv : [2107.05720 \[cs.IR\]](#). URL : <https://arxiv.org/abs/2107.05720>.
- [15] Carlos LASSANCE et al. *SPLADE-v3 : New baselines for SPLADE*. 2024. arXiv : [2403.06789 \[cs.IR\]](#).
- [16] Liang WANG et al. *Multilingual E5 Text Embeddings : A Technical Report*. 2024. arXiv : [2402.05672 \[cs.CL\]](#). URL : <https://arxiv.org/abs/2402.05672>.
- [17] QWEN TEAM. *Qwen3.5 : Towards Native Multimodal Agents*. Fév. 2026. URL : <https://qwen.ai/blog?id=qwen3.5>.
- [18] Abhinand BALACHANDRAN. *MedEmbed : Medical-Focused Embedding Models*. 2024. URL : <https://github.com/abhinand5/MedEmbed>.
- [19] Darren EDGE et al. *From Local to Global : A Graph RAG Approach to Query-Focused Summarization*. 2025. arXiv : [2404.16130 \[cs.CL\]](#). URL : <https://arxiv.org/abs/2404.16130>.
- [20] Stanford NLP. *DSPy*. 2025. URL : <https://dspy.ai>.
- [21] World Health ORGANIZATION. *Classification Internationale des Maladies*. 2022. URL : <https://icd.who.int/en/>.
- [22] INSERM et l'US NATIONAL LIBRARY OF MEDICINE. *Medical Subject Headings (MeSH)*. 2025. URL : <https://mesh.inserm.fr/>.
- [23] Luyu GAO et al. *Precise Zero-Shot Dense Retrieval without Relevance Labels*. 2022. arXiv : [2212.10496 \[cs.IR\]](#). URL : <https://arxiv.org/abs/2212.10496>.

Annexes

A Évaluation avec le NDCG

```
1 def evaluate_retrieval(retrieved_chunks, relevant_chunks, k_values):
2     relevant_keys = set(
3         (chunk.get("filename"), chunk.get("start"), chunk.get("end"))
4         for chunk in relevant_chunks
5     )
6     total_relevant = len(relevant_keys)
7     total_retrieved = len(retrieved_chunks)
8     metrics_by_k = {}
9
10    relevances = []
11    relevant_retrieved = 0
12    for chunk in retrieved_chunks:
13        chunk_key = (chunk.get("filename"), chunk.get("start"), chunk.get("end"))
14    )
15        if chunk_key in relevant_keys:
16            relevances.append(1)
17            relevant_retrieved += 1
18        else:
19            relevances.append(0)
20
21    for k in k_values:
22        retrieved_at_k = relevances[:int(k)]
23        relevant_at_k = sum(retrieved_at_k)
24
25        precision = relevant_at_k / k if k > 0 else 0.0
26        recall = relevant_at_k / total_relevant if total_relevant > 0 else 0.0
27
28        f1 = 0.0
29        if precision + recall > 0:
30            f1 = 2 * (precision * recall) / (precision + recall)
31
32        ndcg = ndcg_at_k(relevances, k) # Metrique de classement
33
34        metrics_by_k[f"k={k}"] = {
35            "precision": precision,
36            "recall": recall,
37            "f1": f1,
38            "ndcg": ndcg
39        }
40    return {"global_metrics": ..., "metrics_by_k": metrics_by_k}
```

Listing IV.1 – Fonction d'évaluation avec calcul du NDCG

B Reranker Hybride

```
1 def multi_rag_reranker(rag_results_list, weights):
2     if not rag_results_list:
```

```

3     return []
4     assert len(rag_results_list) == len(weights)
5     combined = {}
6     for rag_values, weight in zip(rag_results_list, weights):
7         if not rag_values: continue
8         for item in rag_values:
9             chunk_key = (item["filename"], item["start"], item["end"])
10            weighted_score = item["score"] * weight
11            if chunk_key not in combined:
12                combined[chunk_key] = {
13                    "filename": item["filename"],
14                    "start": item["start"],
15                    "end": item["end"],
16                    "content": item["content"],
17                    "score": 0.0
18                }
19            combined[chunk_key]["score"] += weighted_score
20    combined_list = list(combined.values())
21    return sorted(combined_list, key=lambda x: x["score"], reverse=True)

```

Listing IV.2 – Logique de combinaison dynamique de scores (Fusion)

C Résultats HyDE

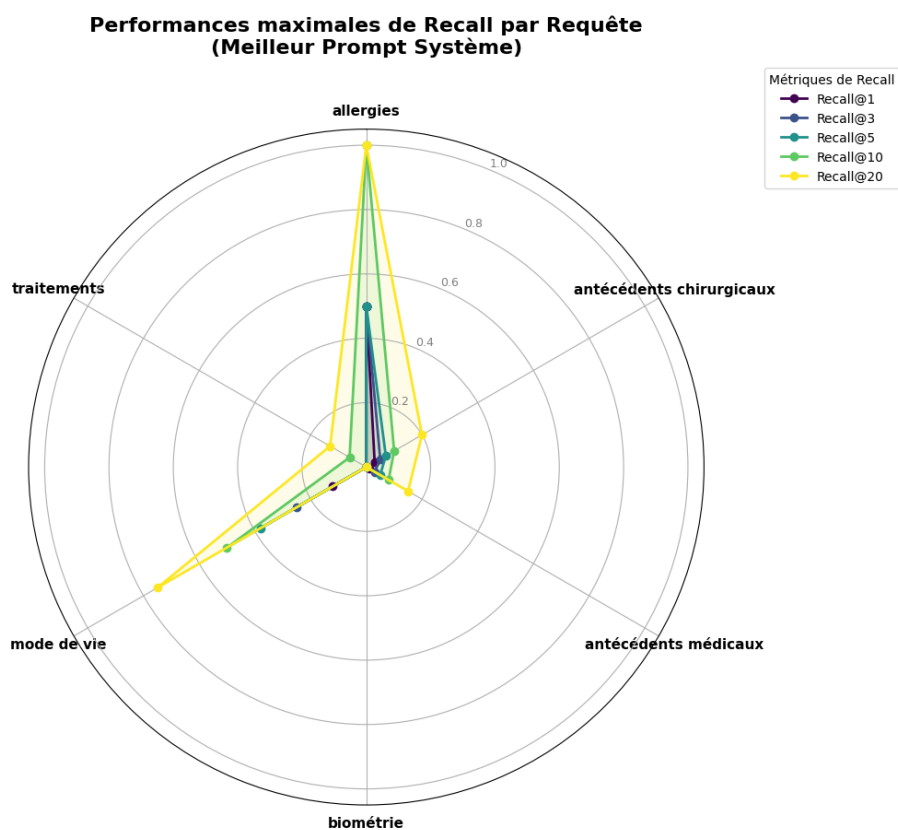


FIGURE IV.1 – Recall@K avec la méthode HyDE, avec un modèle hybride BM25 & sentence-camembert sans reranker, à poids uniformes

On constate de très bons résultats pour la requête *allergies*, même avec une faible proportion de documents récupérés. Cela s'explique par le fait que les allergies sont écrites toujours de la même manière dans les documents du patient : par exemple, une allergie à un médicament sera

toujours indiquée avec le nom du médicament. La création d'un document fictif qui décrit des allergies fictives permet donc de récupérer les documents qui contiennent les termes exactes du document fictif via le modèle sparse, et les documents qui ont un sens proche grâce au modèle dense. Ce type de résultat a également été observé avec DSPy, qui ajoute des termes aux requêtes. On observe le même type de résultat sur la requête *mode de vie*.

Les autres requêtes ont des scores très faibles (recall inférieur à 20%), quelque soit la proportion de chunks récupérés. Une piste d'explication est que la réponse idéale est très variable selon le patient étudié, et à la fois le sens et les termes exacts peuvent aller dans beaucoup de directions. Aussi, un document généré qui correspond à ce qu'on cherche dans le dossier d'un patient est très improbable. Le document influence donc le modèle dense dans une mauvaise direction sémantique, ce qui affaiblit ses performances. Le document fictif n'apporte également que peu au modèle sparse, car il contient peu de termes qui correspondent au dossier patient.

Cette méthode est donc difficile à utiliser dans le contexte de ce projet, et les bons résultats obtenus avec DSPy sont suffisants pour la réécriture de requêtes.